

Kauda Baji: Proposed Rules

For review

M. Naik

July 2026

Why this document

I'm currently working on a hobby project to build an AI agent that plays Kauda Baji. The idea is to build a complete model of the game in code, then design AI agents that can play it. In the future, time permitting, there may even be a robot that plays the game with you in real life.

I've progressed though making models for the board, player, pieces, and moves. The next remaining step is defining the rules.

And this is where I need your help!

The ask is for you to please read this document and provide feedback if you agree or disagree with each rule. Because a computer will be playing using these rules, they must be fully unambiguous. Anything marked as a *question for the family* at the end is genuinely open — I want your opinion.

If the rule set is agreed upon by all parties, it will enable my project progress.

I would be grateful for your time and feedback.

Terms used

Our game is played in Gujarati, but code is typically written in English so I've had to come up with some terms for common concepts. Here are the terms I will use in this document, and their meanings and translations.

Piece

Trans.(ફક્લ, ફક્લ) - One of 6 movable objects belonging to one player.

Tuple

A player's own pieces stacked on the same square and moving together as a unit.

Tuple Size

The number of pieces in a tuple. A tuple of size n is called an n -tuple — a 2-tuple has size 2, a 4-tuple has size 4, etc.

Kaudi

One of six cowrie shells thrown together to generate a move. Plural: *kaudis*.

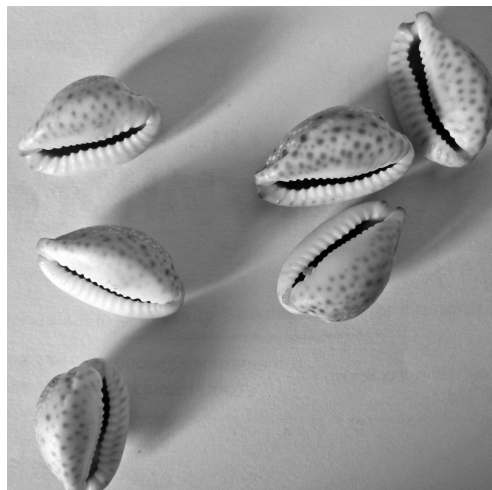
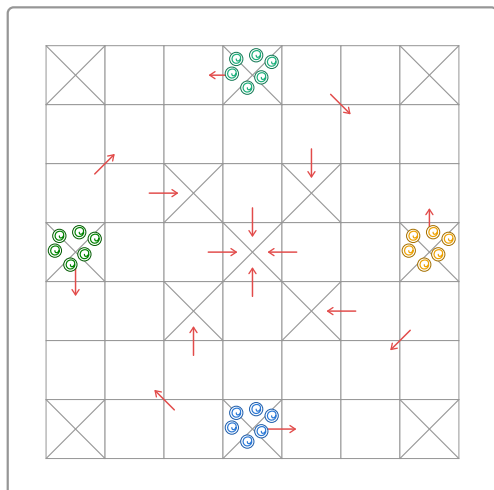
Safe square

A square marked with a corner-to-corner cross. The starting squares and the centre square are safe.

Kill

Removing an opponent's tuple from the board. Killed pieces go back to their owner's starting square and begin their journey again.

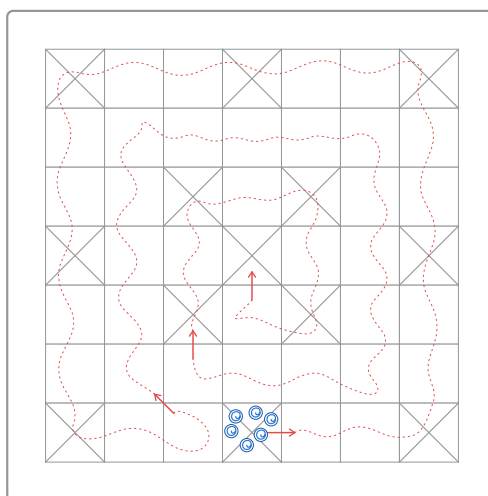
The board and pieces



The board is a 7×7 grid of squares. Four players sit at the four edges; each player owns the edge nearest them. Each player has six pieces of their own colour. Every player's pieces start on the same square: the crossed square in the middle of that player's own edge. This is the player's **starting square**. From there, pieces travel around and inward, and all four players finish on the single square at the very centre of the board.

Twelve squares carry a corner-to-corner cross, and one more sits at the centre — these are the **safe squares**, marked in the diagram above. Every other square is ordinary. What makes safe squares special is covered in the rules below.

Moves are generated by throwing a set of six *kaudis* — cowrie shells — together. Each kaudi can land one of two ways: mouth-up or mouth-down. How a throw translates into a move is covered in a later rules section.



Every piece follows the same route from its starting square to the centre. The route is identical for all four players, just rotated to begin at each player's own edge.

A piece begins on its **starting square** — the crossed square in the middle of the player's edge — and travels anticlockwise around the outer ring of the board. At the **first arrow**, it turns inward onto the first inner band and continues **clockwise** around that band. At the **second arrow**, it turns inward again onto the second inner band, still travelling clockwise. The **third and final arrow** then carries the piece onto the centre square, completing its journey. This is the full route; whether a piece must take each arrow or may skip it is covered in Rules 4 and 5 below.

The objective of the game is to get all six pieces from the starting square to the centre square. The first player to do so wins.

The rules

Rule 1 — Determining the first player

Before play begins, every player throws their kaudis once. Whoever rolls the highest value goes first. Play then proceeds **anticlockwise** around the table from there.

Example: Red throws 4, Blue throws 6, Green throws 2, Yellow throws 12. Yellow goes first (12 beats everything), then play continues anticlockwise from Yellow's seat.

Example: Red throws 5, Blue throws 5, Green throws 6, Yellow throws 3. Green goes first with the single highest value, 6.

Rule 2 — Throwing the kaudis

A turn begins with a throw of all six kaudis. The value of a throw is the number of kaudis that land mouth-up, except that a throw with **no** kaudis mouth-up is worth **12** rather than 0. So a single throw is worth 1, 2, 3, 4, 5, 6, or 12.

A throw of 6 or 12 earns the player an extra throw, and the value of that extra throw is **banked** alongside the first. If a player throws 6 or 12 three times in a row, they forfeit the rest of their turn — none of the banked values may be used.



3 mouth-up → value 3



6 mouth-up → value 6 (bonus: throw again)



0 mouth-up → value 12 (bonus: throw again)

Example: A player throws 6 (a bonus), throws again and gets 4 — since 4 isn't 6 or 12, the turn stops there with two values banked to spend: 6 and 4. A player who instead throws 6, then 12, then 6 has hit a bonus three times in a row and forfeits the whole turn — none of those three values may be used.

Rule 3 — Using a throw to move

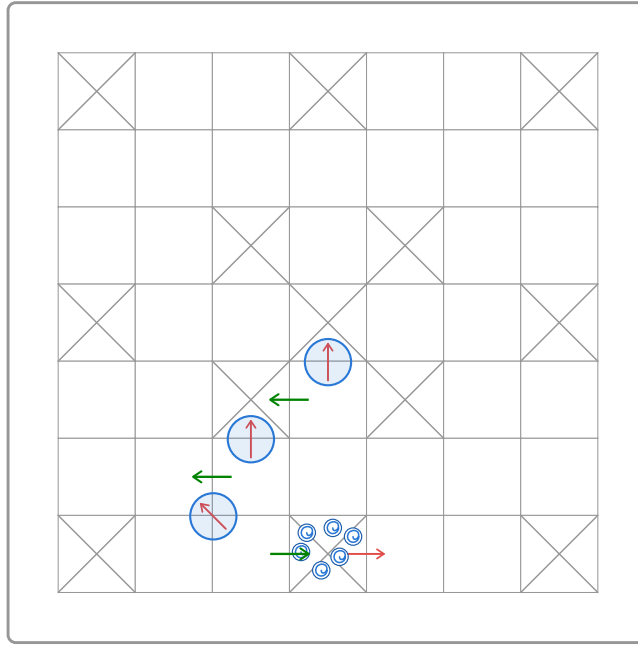
Each value produced during a turn moves one piece (or tuple) that many squares. If a turn produces more than one value, they may all be spent on a single piece (added together), or spread across several different pieces, in any combination the player chooses — but a single value cannot be split between two pieces. A throw of 6 and 3 in the same turn, for example, can move one piece 9 squares in total, or two different pieces 6 and 3 squares each, but the 6 and the 3 can never themselves be divided further.

Example: A turn produces 6, 3, and 2 (three separate values). Legal ways to spend them include: one piece moves 11 (6+3+2); one piece moves 9 (6+3) and a second piece moves 2; or three different pieces move 6, 3, and 2 respectively. Not legal: splitting the 6 into a 4-and-2 to feed two pieces.

Example: A turn produces just one value, 12 (a zero-up throw). The whole 12 must go to a single piece (or tuple) — there's nothing to spread, since there's only the one value.

Rule 4 — Turning at an arrow is optional

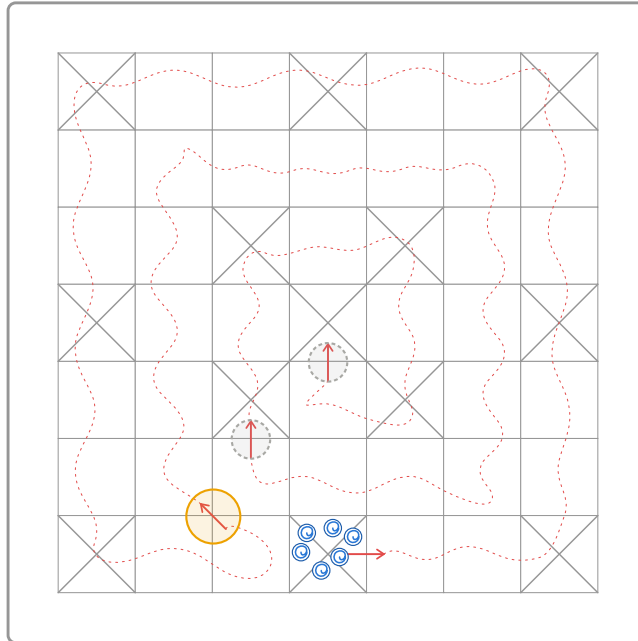
At each of the three arrows described above, a piece is not forced inward. Its owner may choose to follow the arrow onto the next band, or keep the piece moving straight ahead on its current band instead, for strategic reasons. (The first arrow carries one further condition — see Rule 5.)



Shown on the bottom player's path; the same three points apply, rotated, for every player.

Rule 5 — The Markut rule: a kill is required to enter the first inner band

A player may not turn any piece inward at the **first** arrow until that player has killed at least one opposing piece or tuple, at any point earlier in the game. This is a one-off unlock, not a per-piece cost: once a player has made their first kill, *all* of that player's pieces may enter the first inner band freely from then on. No kill is required to enter the second inner band or the centre — the requirement applies only to the first arrow.



Arrow 1 is gated by this rule; arrows 2 and 3 (dashed rings) are not.

Rule 6 — Moving a tuple: the divide rule

A tuple of size n can only be moved using a throw value that n divides without remainder. A 2-tuple needs a throw of 2, 4, 6, or 12; a 4-tuple needs a throw of 4 or 12; a lone piece (a 1-tuple) can move on any throw, since 1 divides everything. A throw value that doesn't divide evenly by a tuple's size simply cannot be used to move that tuple.

Tuple size	Usable throw values (out of 1,2,3,4,5,6,12)
1-tuple (lone piece)	1, 2, 3, 4, 5, 6, 12 — any throw
2-tuple	2, 4, 6, 12
3-tuple	3, 6, 12
4-tuple	4, 12
5-tuple	5 — only an exact throw of 5 works
6-tuple	6, 12

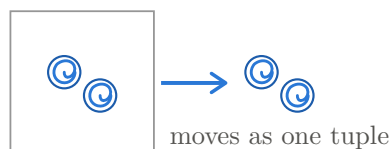
Rule 7 — Tuples form only when a player chooses to move pieces off together

When one of a player's pieces or tuples lands on a square already holding one or more of that **same** player's own pieces or tuples, nothing merges automatically — they simply share the square as separate pieces, exactly as before. A tuple only forms at the moment its owner chooses to move two or more of them off that square **together**, as a single unit; forming and moving happen in the same action, and that move must obey the divide rule (Rule 6) for the combined size. If the pieces are instead moved off individually, or not moved together, no tuple forms.

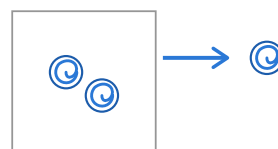
Once formed, a tuple moves, and is killed, together as one unit (see *Tuple* in Terms used).

This does not apply on a player's own **starting square**: pieces waiting there remain single pieces no matter how many share it, and can never be moved off together as a tuple from there. A tuple can only begin forming once a piece has left the starting square.

Moved together: tuple forms



Moved separately: no tuple

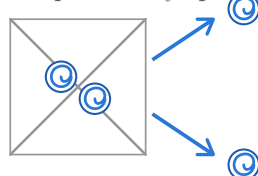


other piece stays, still alone

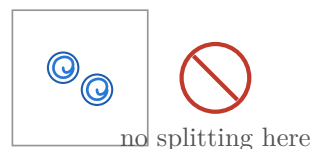
Rule 8 — Tuples may only dissolve on a safe square

A tuple may split back into its individual pieces, but only while it occupies a safe square. Everywhere else on the board, a tuple must move and be killed as a single unit until it next dissolves.

On a safe square: may split



Off a safe square: must move as one



Rule 9 — Landing on an opponent: equal or bigger kills

If your tuple lands on a square holding an opponent's tuple, and your tuple is the **same size or bigger**, the opponent's tuple is killed. Your tuple takes the square; their pieces go back to their start.

Equal size

Before



After



Strictly bigger

Before



After



The same happens if the landing tuple is strictly bigger — a Red 3 landing on a Blue 2 kills it just the same. Equal size is enough; the attacker never needs to outnumber the defender.

Rule 10 — Landing on an opponent: smaller coexists

If your tuple lands on a square holding an opponent's tuple and yours is **strictly smaller**, nothing is killed. Both tuples share the square.



Notice what this means: whenever two opponents share a square, the tuple that was there first is always the bigger one. The smaller tuple is a guest on a stronger tuple's square.

Rule 11 — Leaving a shared square: the bigger tuple kills on exit

When two opponents share a square and the **strictly bigger** tuple moves away, it kills the smaller tuple as it leaves.



The reverse is free: if the *smaller* tuple moves away first, no kill happens — it simply escapes.



So sharing a square is dangerous for the guest. The smaller tuple sits under threat until either it leaves, or the bigger tuple leaves and kills it on the way out.

Rule 12 — Safe squares: no kills, ever

No kill of any kind can happen on a safe square. Any number of tuples, of any sizes, from any players, can share a safe square. Landing there kills nothing, and leaving a shared safe square kills nothing.



Rule 13 — Three or more players on one square: settle each pair separately

When a tuple lands on a square holding tuples from more than one opponent, apply Rules 9 and 10 to **each opponent separately**. The lander kills every opponent tuple that is its size or smaller, and coexists with every opponent tuple that is bigger.

Kills the smaller, spares the bigger



Smaller than both: coexists with both



Exit kills work the same way: a departing tuple kills every strictly smaller opponent tuple it leaves behind, and spares the rest.

Rule 14 — A kill earns a bonus throw

Making a kill — landing on and killing an opponent (Rule 9), or killing on exit (Rule 11 or 13) — earns the killer an immediate extra throw. Besides this and throwing 6 or 12 (Rule 2), nothing else grants a bonus throw.

Rule 15 — Reaching the centre: exact throw required (draft — needs expansion)

Landing on the centre square requires an **exact** throw — a piece may not overshoot it. A player is obliged to use a smaller throw value if one is available and would legally move a piece, but is not obliged to use a throw bigger than what's needed to land a piece on the centre exactly.

What follows from these rules

These are not extra rules — they are consequences of Rules 9–13. I list them because they answer situations that caused arguments in the past.

- **Two opponent tuples of equal size can never share an ordinary square.** Landing with equal size kills (Rule 9), so the situation can never arise in the first place.
- **On a shared ordinary square, the resident is always bigger than the guest.** Coexistence only ever begins with a smaller tuple landing on a bigger one (Rule 10).
- **Big tuples are strong but slow to threaten.** A size-3 tuple can only be killed by a size-3 or bigger landing on it — but it also spends three pieces on one square.
- **A lone piece is never safe on an opponent's square.** As a guest it can be killed the moment the resident moves (Rule 11).
- **A killed tuple's pieces return to the start as singles.** Tuples can't exist on a starting square (Rule 7), so pieces sent back after a kill are forced back into single pieces on arrival. This settles what used to be an open question here: killed pieces always return as singles, never as the tuple they were.

Please mark up anything you disagree with, or reply with the version you remember. Once we agree, this becomes the reference for both the physical board and the computer version I am building.

Kaudi photo: Sodabottle, CC BY-SA 3.0, via Wikimedia Commons.